STUDY TRACK AI BASED

STUDENT STUDY HABIT RECOMMENDER

INFOSYS SPRINGBOARD

Virtual Internship 6.0

Project Guide

Dr. Sumathy S

Team Members

Sneha Rose CJ

Anagha VS

Jismy C Joseph

Krovi Baby Sri Kavya

```
import streamlit as st
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, DBSCAN,
AgglomerativeClustering from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from datetime import datetime, timedelta
import os
import warnings
warnings.filterwarnings('ignore')
# ========== STUDENT LOGIN SYSTEM
========== @st.cache_data
def load_students():
 return pd.read_csv("Students Performance Dataset.csv")

students_df = load_students()

if "logged_in" not in st.session_state:
 st.session_state.logged_in = False

if "student_name" not in st.session_state:
 st.session_state.student_name = ""

# Login page
if not st.session_state.logged_in:

 st.title("►► Student Login")

 first_name = st.text_input("First Name")
 last_name = st.text_input("Last Name")
 student_id = st.text_input("Student ID")

 if st.button("Login"):

 match = students_df[
 (students_df["First_Name"].str.lower() == first_name.lower()) &
(students_df["Last_Name"].str.lower() == last_name.lower()) &
(students_df["Student_ID"].astype(str) == student_id)  ]

 if len(match) > 0:
 st.session_state.logged_in = True
 st.session_state.student_name = first_name + " " + last_name
st.success("Login successful ✅")
 st.rerun()
 else:
 st.error("Invalid login details ❌")
```

```
st.stop()
# ========== PAGE CONFIGURATION ==========
st.set_page_config(
 page_title="Integrated Study Analytics Dashboard",
 layout="wide",
 initial_sidebar_state="expanded",
 page_icon="⁂"
)

# ========== EXTERNAL CSS & HTML INTEGRATION
========== # Load external CSS
def load_css():
 with open("styles.css", "r") as f:
 css = f.read()
 st.markdown(f"<style>{css}</style>", unsafe_allow_html=True)

# Load HTML template
def load_html_template():
 with open("index.html", "r") as f:
 html = f.read()
 return html

# Apply external styles
load_css()

# ========== CUSTOM STYLING ==========
st.markdown("""
<style>

/* ---------- GLOBAL ---------- */
.stApp {
 background: linear-gradient(135deg,#f5f7fa,#e4ecfb);
 }

/* ---------- HEADER ---------- */
.main-header {
 background: linear-gradient(135deg,#667eea,#764ba2);
padding:25px;
 border-radius:18px;
 color:white;
 margin-bottom:20px;
 box-shadow:0 10px 25px rgba(0,0,0,0.1);
 }

.main-title {
 font-size:32px;
 font-weight:700;
 }
```

```
.subtitle {
 font-size:15px;
 opacity:0.9;
 }

/* ---------- CONTENT CARD ---------- */
.content-card {
 background:white;
 padding:20px;
 border-radius:15px;
 box-shadow:0 4px 15px rgba(0,0,0,0.05);
margin-bottom:20px;
 }

/* ---------- METRIC CARDS ---------- */
.metric-card {
 background: linear-gradient(135deg,#667eea,#764ba2);
padding:20px;
 border-radius:15px;
 color:white;
 text-align:center;
 box-shadow:0 6px 15px rgba(0,0,0,0.08); }

.metric-value {
 font-size:26px;
 font-weight:bold;
 }

.metric-label {
 font-size:14px;
 }

/* ---------- SIDEBAR ---------- */
[data-testid="stSidebar"] {
 background: linear-gradient(180deg,#667eea,#764ba2); }

[data-testid="stSidebar"] * {
 color:white !important;
 }

/* ---------- BUTTON ---------- */
.stButton>button {
 background: linear-gradient(135deg,#667eea,#764ba2);
color:white;
 border:none;
 border-radius:10px;
 height:40px;
 font-weight:600;
 }
```

```
.stButton>button:hover {
 transform: translateY(-2px);
 box-shadow:0 5px 12px rgba(0,0,0,0.15);
 }

/* ---------- CHART CARD ---------- */
.chart-card {
 background:white;
 padding:15px;
 border-radius:15px;
 box-shadow:0 2px 8px rgba(0,0,0,0.05);
 }

/* ---------- DATAFRAME ---------- */
[data-testid="stDataFrame"] {
 border-radius:12px;
 overflow:hidden;
 }

/* ---------- TOOL CARD ---------- */
.tool-card {
 background:white;
 padding:15px;
 border-radius:12px;
 box-shadow:0 3px 10px rgba(0,0,0,0.05);
 text-align:center;
 }

/* ---------- RECOMMENDATION CARD ---------- */
.recommendation-card {
 background:linear-gradient(135deg,#eef2ff,#f8f9ff);
 padding:20px;
 border-radius:15px;
 border-left:5px solid #667eea;
 }

</style>
""", unsafe_allow_html=True)

# ========== DATA LOADING & CACHING
 ========== @st.cache_data
def load_student_data():
 try:
 if os.path.exists("Students Performance Dataset.csv"):
 df = pd.read_csv("Students Performance Dataset.csv")  else:
 df = pd.read_csv(r"D:\Desktop\app\Students Performance Dataset.csv")
return df
 except:
 st.error("Could not load Students Performance Dataset.csv")
```

```
return None

@st.cache_data
def load_clustered_data():
 try:
 df = pd.read_csv("students_with_clusters.csv")
 return df
 except:
 return None

@st.cache_data
def load_study_logs():
 if os.path.exists("study_logs.csv"):
 return pd.read_csv("study_logs.csv")
 else:
 return pd.DataFrame(columns=["date", "study_duration", "study_time", "subject",
"distraction_level", "quiz_score", "predicted_cluster"])

def save_study_log(log_data):
 logs_df = load_study_logs()
 logs_df = pd.concat([logs_df, pd.DataFrame([log_data])], ignore_index=True)
logs_df.to_csv("study_logs.csv", index=False)
 st.cache_data.clear()

# ========== HELPER FUNCTIONS ==========
def prepare_clustering_data(df, selected_features):
 """Prepare data for clustering"""
 X = df[selected_features].copy()
 before_rows = X.shape[0]
 X = X.dropna()
 after_rows = X.shape[0]

 scaler = StandardScaler()
 X_scaled = scaler.fit_transform(X)

 return X_scaled, scaler, before_rows - after_rows

def run_clustering(df, X_scaled, n_clusters, eps, min_samples,
selected_features):  """Run all clustering algorithms"""
 results = {}

 # KMeans
 kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
 labels_km = kmeans.fit_predict(X_scaled)
 results['KMeans'] = {'labels': labels_km, 'silhouette': silhouette_score(X_scaled, labels_km)}
 # Agglomerative
 agg = AgglomerativeClustering(n_clusters=n_clusters)
 labels_ag = agg.fit_predict(X_scaled)
 results['Agglomerative'] = {'labels': labels_ag, 'silhouette': silhouette_score(X_scaled, labels_ag)}
 # DBSCAN
```

```
 dbscan = DBSCAN(eps=eps, min_samples=min_samples)
 labels_db = dbscan.fit_predict(X_scaled)
 sil_db = None
 if len(set(labels_db)) > 1 and not set(labels_db) == {-1}:
 mask = labels_db != -1
 sil_db = silhouette_score(X_scaled[mask], labels_db[mask])
results['DBSCAN'] = {'labels': labels_db, 'silhouette': sil_db}
 return results

# ========== CLUSTER BEHAVIOR MAPPINGS
========== CLUSTER_BEHAVIOR = {
 0: "High Focus Learners",
 1: "Low Focus Learners",
 2: "Distracted Learners",
 3: "Visual Learners"
 }

CLUSTER_STUDY_PARAMS = {
 0: {
 "time_slots": ["8:00 AM - 10:00 AM", "2:00 PM - 4:00 PM"],
"duration": 25,
 "breaks": "5 min every 25 min",
 "effectiveness": 85,
 "method": "Focused deep work sessions"
 },
 1: {
 "time_slots": ["10:00 AM - 12:00 PM", "4:00 PM - 6:00 PM"],
"duration": 20,
 "breaks": "10 min every 20 min",
 "effectiveness": 70,
 "method": "Short, frequent study sessions"
 },
 2: {
 "time_slots": ["7:00 AM - 9:00 AM", "7:00 PM - 9:00 PM"],
"duration": 15,
 "breaks": "15 min every 15 min",
 "effectiveness": 60,
 "method": "Distraction-free environment setup"
 },
 3: {
 "time_slots": ["9:00 AM - 11:00 AM", "3:00 PM - 5:00 PM"],
"duration": 30,
 "breaks": "5 min every 30 min",
 "effectiveness": 80,
 "method": "Visual aids and multimedia learning"
 }
 }

BEHAVIOR_TOOLS = {
 "High Focus Learners": ["Mock test platforms", "Advanced study timers"],
```

```
"Low Focus Learners": ["Pomodoro Timer", "Focus music apps"],
"Distracted Learners": ["Site Blocker", "Noise-cancelling headphones"],
"Visual Learners": ["Digital Notes", "Mind mapping tools"] }

# ========== MAIN APPLICATION ==========
# Custom Header with light theme
st.markdown("""
<div class="main-header">
<h1 class="main-title">►► Recommending Study Habits Based on Student Behavior</h1> <p
class="subtitle">AI-Based Student Behavior Analysis & Personalized Study Recommendation
System</p>
</div>
""", unsafe_allow_html=True)
col1, col2 = st.columns([8,2])
with col2:
 if st.button("Logout"):
 st.session_state.logged_in = False
 st.rerun()
# Create tabs with enhanced styling
tab1, tab2, tab3, tab4, tab5 = st.tabs([
 "►► Data Analysis",
 "►► Student Clustering",
 "►► Study Plans",
 "►► Study Tracker",
 "►► Progress Analytics"
])

# ========== TAB 1: DATA ANALYSIS ==========
with tab1:
 st.markdown('<div class="content-card fade-in">', unsafe_allow_html=True)
st.header("►► Data Preprocessing & Exploratory Data Analysis")
 st.markdown("</div>", unsafe_allow_html=True)

 df = load_student_data()

 if df is not None:
 # Enhanced Metrics Cards
 st.markdown('<div class="fade-in">', unsafe_allow_html=True)
 col1, col2, col3, col4 = st.columns(4)

 with col1:
 st.markdown(f"""
 <div class="metric-card">
 <div class="metric-value">{df.shape[0]:,}</div>
 <div class="metric-label">Total Students</div>
 </div>
 """, unsafe_allow_html=True)

 with col2:
 st.markdown(f"""
```

```
<div class="metric-card">
<div class="metric-value">{df.shape[1]}</div>
<div class="metric-label">Total Features</div>
</div>
""", unsafe_allow_html=True)

with col3:
missing_count = df.isnull().sum().sum()
st.markdown(f"""
<div class="metric-card">
<div class="metric-value">{missing_count}</div>
<div class="metric-label">Missing Values</div>
</div>
""", unsafe_allow_html=True)

with col4:
numeric_count = df.select_dtypes(include=[np.number]).shape[1]
st.markdown(f"""
<div class="metric-card">
<div class="metric-value">{numeric_count}</div>
<div class="metric-label">Numeric Features</div>
</div>
""", unsafe_allow_html=True)
st.markdown('</div>', unsafe_allow_html=True)

st.markdown("---")

# Dataset Overview with enhanced cards
st.markdown('<div class="content-card fade-in">', unsafe_allow_html=True)
st.subheader("►► Dataset Overview")

col_left, col_right = st.columns(2)

with col_left:
st.markdown("**First 5 Rows**")
st.markdown('<div class="dataframe-container">', unsafe_allow_html=True)
st.dataframe(df.head(), use_container_width=True)
st.markdown('</div>', unsafe_allow_html=True)

with col_right:
st.markdown("**Dataset Information**")
st.markdown(f"**Shape:** {df.shape[0]} rows × {df.shape[1]} columns")
st.markdown(f"**Data Types:** {df.dtypes.nunique()} unique types")
missing = df.isnull().sum()
if missing.sum() > 0:
st.markdown("**Missing Values by Column:**")
missing_df = missing[missing > 0].reset_index()
missing_df.columns = ['Column', 'Missing Count']
st.markdown('<div class="dataframe-container">', unsafe_allow_html=True)
st.dataframe(missing_df, use_container_width=True)
```

```
st.markdown('</div>', unsafe_allow_html=True)
else:
st.success("✅ No missing values detected!")
st.markdown('</div>', unsafe_allow_html=True)

st.markdown("---")

# Statistical Summary
st.markdown('<div class="content-card fade-in">', unsafe_allow_html=True)
st.subheader("►► Statistical Summary")
st.markdown('<div class="dataframe-container">', unsafe_allow_html=True)
st.dataframe(df.describe().T, use_container_width=True)  st.markdown('</div>',
unsafe_allow_html=True)
st.markdown('</div>', unsafe_allow_html=True)

st.markdown("---")

# Enhanced Visualizations
st.markdown('<div class="content-card fade-in">', unsafe_allow_html=True)
st.subheader("►► Data Visualizations")

# Row 1: Scatter plot and Grade distribution
col1, col2 = st.columns(2)

with col1:
st.markdown("**Study Hours vs Quiz Performance**")
scatter_fig = px.scatter(
df,
x="Study_Hours_per_Week",
y="Quizzes_Avg",
color="Grade",
title="",
trendline="ols",
hover_data=["Total_Score"],
color_discrete_sequence=px.colors.qualitative.Set1
)
scatter_fig.update_layout(
plot_bgcolor='rgba(0,0,0,0)',
paper_bgcolor='rgba(0,0,0,0)',
font=dict(size=12)
)
st.plotly_chart(scatter_fig, use_container_width=True)
with col2:
st.markdown("**Grade Distribution**")
grade_dist = df["Grade"].value_counts()
donut_fig = px.pie(
values=grade_dist.values,
names=grade_dist.index,
title="",
hole=0.6,
```

```
 color_discrete_sequence=px.colors.qualitative.Pastel  )
 donut_fig.update_layout(
 plot_bgcolor='rgba(0,0,0,0)',
 paper_bgcolor='rgba(0,0,0,0)',
 font=dict(size=12)
 )

 st.plotly_chart(donut_fig, use_container_width=True)

 # Row 2: Correlation heatmap and score distribution
col1, col2 = st.columns(2)

 with col1:
 st.markdown("**Feature Correlation Matrix**")  corr
= df.corr(numeric_only=True)
 heatmap_fig = go.Figure(
 data=go.Heatmap(
 z=corr.values,
 x=corr.columns,
 y=corr.columns,
 colorscale="RdBu_r",
 zmin=-1, zmax=1
 )
 )
 heatmap_fig.update_layout(
 height=500,
 plot_bgcolor='rgba(0,0,0,0)',
 paper_bgcolor='rgba(0,0,0,0)',
 font=dict(size=10)
 )
 st.plotly_chart(heatmap_fig, use_container_width=True)
 with col2:
 st.markdown("**Total Score Distribution**")
hist_fig = px.histogram(
 df,
 x="Total_Score",
 nbins=25,
 title="",
 color_discrete_sequence=["#667eea"],
marginal="box"
 )
 hist_fig.update_layout(
 plot_bgcolor='rgba(0,0,0,0)',
 paper_bgcolor='rgba(0,0,0,0)',
 font=dict(size=12)
 )
 st.plotly_chart(hist_fig, use_container_width=True)
 # Row 3: Box plots
 col1, col2 = st.columns(2)
```

```
 with col1:
 st.markdown("**Performance by Grade**")
box_fig = px.box(
 df,
 x="Grade",
 y="Total_Score",
 color="Grade",
 title="",
 color_discrete_sequence=px.colors.qualitative.Set2
 )
 box_fig.update_layout(
 plot_bgcolor='rgba(0,0,0,0)',
 paper_bgcolor='rgba(0,0,0,0)',
 font=dict(size=12)
 )
 st.plotly_chart(box_fig, use_container_width=True)

 with col2:
 st.markdown("**Study Hours by Grade**")
 box_fig2 = px.box(
 df,
 x="Grade",
 y="Study_Hours_per_Week",
 color="Grade",
 title="",
 color_discrete_sequence=px.colors.qualitative.Set3
 )
 box_fig2.update_layout(
 plot_bgcolor='rgba(0,0,0,0)',
 paper_bgcolor='rgba(0,0,0,0)',
 font=dict(size=12)
 )
 st.plotly_chart(box_fig2, use_container_width=True)

 st.markdown('</div>', unsafe_allow_html=True)

# ========== TAB 2: STUDENT CLUSTERING ==========
with tab2:
 st.markdown('<div class="content-card fade-in">', unsafe_allow_html=True)
st.header("►► Student Performance Clustering Analysis")
 st.markdown("Discover patterns in student behavior and performance through advanced machine
learning clustering algorithms.")
 st.markdown('</div>', unsafe_allow_html=True)

 df = load_student_data()

 if df is not None:
 # Enhanced Sidebar
 with st.sidebar:
```

```
st.markdown('<div class="sidebar-content">', unsafe_allow_html=True)
st.header("⚙️ Clustering Controls")

DEFAULT_FEATURES = [
"Attendance (%)",
"Midterm_Score",
"Final_Score",
"Assignments_Avg",
"Quizzes_Avg",
"Participation_Score",
"Projects_Score",
"Total_Score",
"Study_Hours_per_Week",
]

n_clusters = st.slider("Number of clusters", 2, 10, 4, help="Choose the number of student groups to identify")
eps = st.slider("DBSCAN eps", 0.1, 5.0, 0.5, step=0.1, help="DBSCAN neighborhood distance")
min_samples = st.slider("DBSCAN min samples", 2, 20, 5, help="Minimum samples for DBSCAN core points")

available_features = [f for f in DEFAULT_FEATURES if f in df.columns]  features = st.multiselect("Select features for clustering", available_features, default=available_features[:6], help="Choose which student metrics to use for grouping")
st.markdown("---")
run_button = st.button("🚀 Run Clustering Analysis", type="primary", use_container_width=True)
st.markdown('</div>', unsafe_allow_html=True)

if run_button and features:
# Prepare data
X_scaled, scaler, dropped_rows = prepare_clustering_data(df, features)
if dropped_rows > 0:
st.warning(f"⚠️ Dropped {dropped_rows} rows with missing values")
# Run clustering
results = run_clustering(df, X_scaled, n_clusters, eps, min_samples, features)
# Enhanced Quality Metrics
st.markdown('<div class="content-card fade-in">', unsafe_allow_html=True)
st.subheader("📊 Clustering Quality Metrics")

col1, col2, col3 = st.columns(3)

with col1:
kmeans_score = results['KMeans']['silhouette']
st.markdown(f"""
<div class="metric-card">
<div class="metric-value">{kmeans_score:.3f}</div>
<div class="metric-label">K-Means Silhouette</div>
```

```
</div>
""", unsafe_allow_html=True)

with col2:
agg_score = results['Agglomerative']['silhouette']
st.markdown(f"""
<div class="metric-card">
<div class="metric-value">{agg_score:.3f}</div>
<div class="metric-label">Agglomerative Silhouette</div>
</div>
""", unsafe_allow_html=True)

with col3:
db_score = results['DBSCAN']['silhouette']
st.markdown(f"""
<div class="metric-card">
<div class="metric-value">{f"{db_score:.3f}" if db_score is not None else
"N/A"}</div>
<div class="metric-label">DBSCAN Silhouette</div>  </div>
""", unsafe_allow_html=True)

st.markdown('</div>', unsafe_allow_html=True)

st.markdown("---")

# Enhanced PCA Visualization
st.markdown('<div class="content-card fade-in">', unsafe_allow_html=True)
st.subheader("►► Cluster Visualizations (PCA Projection)")

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

col1, col2, col3 = st.columns(3)

methods = ['KMeans', 'Agglomerative', 'DBSCAN']
fig_list = []

for i, method in enumerate(methods):
labels = results[method]['labels']
# Convert labels to strings for better coloring
labels_str = [f"Cluster {int(label)}" for label in labels]

fig = px.scatter(
x=X_pca[:, 0],
y=X_pca[:, 1],
color=labels_str,
title=f"{method} Clustering",
labels={"color": "Student Group"},
color_discrete_sequence=px.colors.qualitative.Set1
)
```

```
fig.update_layout(
height=400,
plot_bgcolor='rgba(0,0,0,0)',
paper_bgcolor='rgba(0,0,0,0)',
font=dict(size=11)
)
fig.update_traces(marker=dict(size=8, opacity=0.7))
fig_list.append(fig)

with col1:
st.plotly_chart(fig_list[0], use_container_width=True)
with col2:
st.plotly_chart(fig_list[1], use_container_width=True)
with col3:
st.plotly_chart(fig_list[2], use_container_width=True)

st.markdown('</div>', unsafe_allow_html=True)

st.markdown("---")

# Enhanced Data Export
st.markdown('<div class="content-card fade-in">', unsafe_allow_html=True)
st.subheader("►► Export Clustered Data")

# Store clusters in dataframe
df_clustered = df.copy()
df_clustered['Cluster_KMeans'] = results['KMeans']['labels']
df_clustered['Cluster_Agglomerative'] = results['Agglomerative']['labels']
df_clustered['Cluster_DBSCAN'] = results['DBSCAN']['labels']

col1, col2 = st.columns([1, 3])
with col1:
if st.button("►► Save Clustered Data", type="primary", use_container_width=True):
df_clustered.to_csv("students_with_clusters.csv", index=False)  st.success("✅ Clustered
data saved successfully!")

with col2:
st.markdown('<div class="dataframe-container">', unsafe_allow_html=True)
st.dataframe(df_clustered.head(), use_container_width=True)
st.markdown('</div>', unsafe_allow_html=True)

st.markdown('</div>', unsafe_allow_html=True)

# Enhanced Cluster Profiles
st.markdown('<div class="content-card fade-in">', unsafe_allow_html=True)
st.subheader("►► Cluster Profiles & Analysis")

profile_tabs = st.tabs(["K-Means Analysis", "Hierarchical Analysis", "DBSCAN Analysis"])
for tab_idx, method in enumerate(['Cluster_KMeans', 'Cluster_Agglomerative',
'Cluster_DBSCAN']):
```

```
 with profile_tabs[tab_idx]:
 if method in df_clustered.columns:
 profile = df_clustered.groupby(method)[features].mean().round(3)
st.dataframe(profile, use_container_width=True)

# ========== TAB 3: PERSONALIZED STUDY PLANS
========== with tab3:
 st.header("►► Personalized Study Plans")

 clustered_df = load_clustered_data()

 if clustered_df is not None:
 # Student selection
 if "Student_ID" in clustered_df.columns:
 student_options = [f"{row['Student_ID']} - {row.get('First_Name', 'Unknown')}
 {row.get('Last_Name', '')}"
 for _, row in clustered_df.iterrows()]
 selected_student = st.selectbox("Select Student", student_options)

 if selected_student:
 student_id = selected_student.split(" - ")[0]
 student_data = clustered_df[clustered_df["Student_ID"] == student_id].iloc[0]  cluster
= int(student_data.get("Cluster_KMeans", 0))
 behavior = CLUSTER_BEHAVIOR.get(cluster, "Unknown")
 params = CLUSTER_STUDY_PARAMS.get(cluster, CLUSTER_STUDY_PARAMS[0])  tools =
BEHAVIOR_TOOLS.get(behavior, ["General tools"])

 # Student Profile
 col1, col2, col3 = st.columns(3)

 with col1:
 st.markdown("### Student Profile")
 st.write(f"**ID:** {student_id}")
 st.write(f"**Cluster:** {cluster}")
 st.write(f"**Behavior Type:** {behavior}")

 with col2:
 st.markdown("### Academic Performance")
 st.write(f"**Total Score:** {student_data.get('Total_Score', 0):.2f}")
st.write(f"**Grade:** {student_data.get('Grade', 'N/A')}")
 st.write(f"**Attendance:** {student_data.get('Attendance (%)', 0):.1f}%")
 with col3:
 st.markdown("### Well-being Metrics")
 st.write(f"**Stress Level:** {student_data.get('Stress_Level (1-10)', 0)}/10")
st.write(f"**Sleep Hours:** {student_data.get('Sleep_Hours_per_Night', 0):.1f}")
st.write(f"**Study Hours/Week:** {student_data.get('Study_Hours_per_Week',  0):.1f}")

 st.markdown("---")

 # Study Plan
```

```
st.markdown("### ►► Recommended Study Plan")

col1, col2 = st.columns([2, 1])

with col1:
st.markdown(f"""
**Study Method:** {params['method']}

**Optimal Study Time Slots:**
""")
for slot in params['time_slots']:
st.write(f"• {slot}")

st.write(f"**Session Duration:** {params['duration']} minutes")
st.write(f"**Break Schedule:** {params['breaks']}")

with col2:
st.markdown(f"""
### Effectiveness Score

**{params['effectiveness']}%**

Improvement potential
""")

st.markdown("---")

# Performance Breakdown
st.subheader("Performance Breakdown by Component")

components = ["Midterm_Score", "Final_Score", "Assignments_Avg", "Quizzes_Avg",
"Participation_Score", "Projects_Score"]
component_labels = ["Midterm", "Final", "Assignments", "Quizzes", "Participation",
"Projects"]
component_values = [student_data.get(comp, 0) for comp in components]
fig_perf = px.bar(
x=component_labels,
y=component_values,
title="Performance Across Components",
color=component_values,
color_continuous_scale="Viridis"
)
st.plotly_chart(fig_perf, use_container_width=True)

st.markdown("---")

# Weekly Schedule
st.subheader("Weekly Study Schedule")

daily_hours = {
```

```
0: [2, 2, 1.5, 2, 2, 1, 1],
1: [1, 1, 1, 1, 1, 0.5, 0.5],
2: [1.5, 1, 1, 1.5, 1, 2, 1],
3: [2, 1.5, 2, 1.5, 2, 1, 1]
}.get(cluster, [1.5]*7)

days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

fig_weekly = px.bar(
x=days,
y=daily_hours,
title="Recommended Study Hours per Day",
labels={"y": "Hours", "x": "Day"}
)
st.plotly_chart(fig_weekly, use_container_width=True)

st.markdown("---")

# Recommended Tools
st.subheader("►► Recommended Study Tools")

cols = st.columns(len(tools))
for i, tool in enumerate(tools):
with cols[i]:
st.markdown(f"""
<div class="tool-card">
<h4>►► {tool}</h4>
<p>Recommended for {behavior.lower()}</p>
</div>
""", unsafe_allow_html=True)

# ========== TAB 4: STUDY TRACKER ==========
with tab4:
st.header("►► Study Behavior Tracker")

clustered_df = load_clustered_data()

if clustered_df is not None:
col1, col2 = st.columns([1, 1])

with col1:
st.subheader("Log Your Study Session")

with st.form("study_form"):
date = st.date_input("Date", datetime.now().date())
study_duration = st.slider("Study Duration (minutes)", 15, 300, 60)  study_time =
st.selectbox("Study Time", ["Morning", "Afternoon", "Evening"])  subject =
st.text_input("Subject Studied", "")
distraction_level = st.selectbox("Distraction Level", ["None", "Low", "Medium",
"High"])
```

```
quiz_score = st.slider("Recent Quiz Score (%)", 0, 100, 75)
submitted = st.form_submit_button("Save & Get Recommendations")
if submitted:
# Create log entry
log_data = {
"date": date,
"study_duration": study_duration,
"study_time": study_time,
"subject": subject,
"distraction_level": distraction_level,
"quiz_score": quiz_score,
"predicted_cluster": 0
 }

save_study_log(log_data)
st.success("✅ Study session logged successfully!")

# Get cluster-based recommendations
if quiz_score >= 80:
cluster = 0 # High focus
elif quiz_score >= 60:
cluster = 3 # Visual
elif quiz_score >= 40:
cluster = 1 # Low focus
else:
cluster = 2 # Distracted

behavior = CLUSTER_BEHAVIOR.get(cluster, "Unknown")  params =
CLUSTER_STUDY_PARAMS.get(cluster,
CLUSTER_STUDY_PARAMS[0])

st.markdown("---")
st.subheader("►► Your Personalized Recommendations")

st.markdown(f"""
<div class="recommendation-card">
<h3>Study Profile: {behavior}</h3>
<p><strong>Optimal Study Times:</strong> {', '.join(params['time_slots'])}</p>
<p><strong>Recommended Duration:</strong> {params['duration']} minutes per  session</p>
<p><strong>Break Schedule:</strong> {params['breaks']}</p>  <p><strong>Study
Method:</strong> {params['method']}</p>  <p><strong>Expected
Effectiveness:</strong> {params['effectiveness']}%  improvement</p>
</div>
""", unsafe_allow_html=True)

with col2:
st.subheader("Recent Study Sessions")
logs_df = load_study_logs()
```

```
if not logs_df.empty:
# Convert date to datetime
logs_df['date'] = pd.to_datetime(logs_df['date'])
recent_logs = logs_df.sort_values('date', ascending=False).head(10)
st.dataframe(recent_logs, use_container_width=True)

# Summary stats
st.markdown("---")
st.write("**Study Statistics**")
total_hours = recent_logs['study_duration'].sum() / 60
avg_score = recent_logs['quiz_score'].mean()
col1, col2, col3 = st.columns(3)
col1.metric("Total Study Hours", f"{total_hours:.1f}")
col2.metric("Avg Quiz Score", f"{avg_score:.1f}%")
col3.metric("Total Sessions", len(recent_logs))
else:
st.info("No study logs yet. Start logging your sessions!")

st.markdown("---")

# ========== ADMIN PANEL ==========
with st.expander("►► Admin Panel", expanded=False):
st.markdown("### System Administration & Data Management")

admin_col1, admin_col2 = st.columns(2)

with admin_col1:
st.markdown("#### ►► Data Export")

# Export study logs
logs_df = load_study_logs()
if not logs_df.empty:
csv_logs = logs_df.to_csv(index=False)
st.download_button(
label="►► Download Study Logs (CSV)",
data=csv_logs,
file_name=f"study_logs_backup_{datetime.now().strftime('%Y%m%d')}.csv",
mime="text/csv",
help="Download all study session logs"
)

# Export clustered data
clustered_df = load_clustered_data()
if clustered_df is not None:
csv_clusters = clustered_df.to_csv(index=False)
st.download_button(
label="►► Download Cluster Data (CSV)",
data=csv_clusters,

file_name=f"students_clusters_backup_{datetime.now().strftime('%Y%m%d')}.csv"
```

```
,  mime="text/csv",
 help="Download all student clustering data"
 )

 # Export student dataset
 df = load_student_data()
 if df is not None:
 csv_data = df.to_csv(index=False)
 st.download_button(
 label="►► Download Student Dataset (CSV)",
 data=csv_data,
 file_name=f"students_dataset_backup_{datetime.now().strftime('%Y%m%d')}.csv",
mime="text/csv",
 help="Download original student performance data"
 )

 with admin_col2:
 st.markdown("#### ►► Data Management")

 # View system statistics
 st.markdown("**System Statistics:**")
 col_stat1, col_stat2, col_stat3 = st.columns(3)

 logs_df = load_study_logs()
 if not logs_df.empty:
 col_stat1.metric("Study Logs", len(logs_df))
 else:
 col_stat1.metric("Study Logs", 0)

 clustered_df = load_clustered_data()
 if clustered_df is not None:
 col_stat2.metric("Clustered Students", len(clustered_df))  else:
 col_stat2.metric("Clustered Students", 0)

 df = load_student_data()
 if df is not None:
 col_stat3.metric("Total Students", len(df))
 else:
 col_stat3.metric("Total Students", 0)

 st.markdown("---")
 st.markdown("**Clear Data Actions:**")

 # Delete study logs
 if st.button("►► Clear All Study Logs", help="Delete all recorded study sessions"):  try:
 import os
 if os.path.exists("study_logs.csv"):
 os.remove("study_logs.csv")
 st.cache_data.clear()
 st.success("✅ All study logs cleared successfully")
```

```
except Exception as e:
st.error(f"❌ Error clearing logs: {e}")

# Reset cluster data
if st.button("►► Reset Cluster Data", help="Clear students_with_clusters.csv"):  try:
import os
if os.path.exists("students_with_clusters.csv"):
os.remove("students_with_clusters.csv")
st.cache_data.clear()
st.success("✅ Cluster data reset successfully")
except Exception as e:
st.error(f"❌ Error resetting clusters: {e}")

st.markdown("---")
st.markdown("#### ►► Configuration Info")

config_col1, config_col2 = st.columns(2)

with config_col1:
st.markdown("**Active Clusters:**")
for cluster_id, cluster_name in CLUSTER_BEHAVIOR.items():
st.write(f"• Cluster {cluster_id}: {cluster_name}")

with config_col2:
st.markdown("**Applied Study Methods:**")
for cluster_id, params in CLUSTER_STUDY_PARAMS.items():  st.write(f"•
{CLUSTER_BEHAVIOR[cluster_id]}: {params['duration']}min sessions")
st.markdown("---")
st.info("ℹ️ **Admin Panel Info:** Use this section to manage data, download backups, and
view system statistics. Always backup data before clearing.")

# ========== TAB 5: PROGRESS ANALYTICS ==========
with tab5:
st.header("►► Progress Analytics Dashboard")

clustered_df = load_clustered_data()
logs_df = load_study_logs()

if clustered_df is not None and not logs_df.empty:
# Convert date
logs_df['date'] = pd.to_datetime(logs_df['date'])

col1, col2, col3, col4 = st.columns(4)

with col1:
st.metric("Total Sessions", len(logs_df))
with col2:
total_hours = logs_df['study_duration'].sum() / 60
st.metric("Total Study Hours", f"{total_hours:.1f}")
with col3:
```

```
avg_score = logs_df['quiz_score'].mean()
st.metric("Avg Quiz Score", f"{avg_score:.1f}%")
with col4:
max_score = logs_df['quiz_score'].max()
st.metric("Best Score", f"{max_score:.0f}%")

st.markdown("---")

# Study Progress Over Time
st.subheader("Study Progress Over Time")

col1, col2 = st.columns(2)

with col1:
fig_score = px.line(
logs_df.sort_values('date'),
x='date',
y='quiz_score',
title='Quiz Score Progress',
markers=True
)
st.plotly_chart(fig_score, use_container_width=True)
with col2:
daily_study = logs_df.groupby('date')['study_duration'].sum() / 60
fig_study = px.bar(
x=daily_study.index,
y=daily_study.values,
title='Daily Study Hours',
labels={"x": "Date", "y": "Hours"}
)
st.plotly_chart(fig_study, use_container_width=True)
st.markdown("---")

# Study Distribution
col1, col2 = st.columns(2)

with col1:
st.subheader("Study Sessions by Time of Day")
time_dist = logs_df['study_time'].value_counts()
fig_time = px.pie(
values=time_dist.values,
names=time_dist.index,
title="Study Sessions by Time"
)
st.plotly_chart(fig_time, use_container_width=True)
with col2:
st.subheader("Distraction Level Distribution")
distract_dist = logs_df['distraction_level'].value_counts()
fig_distract = px.bar(
x=distract_dist.index,
```

```
y=distract_dist.values,
title="Distraction Levels",
labels={"x": "Distraction Level", "y": "Count"}  )
st.plotly_chart(fig_distract, use_container_width=True)
st.markdown("---")

# Subject-wise Analysis
st.subheader("Subject-wise Analysis")

subjects = logs_df[logs_df['subject'].notna()]['subject'].unique()
if len(subjects) > 0:
subject_data = []
for subject in subjects:
subject_logs = logs_df[logs_df['subject'] == subject]
subject_data.append({
'Subject': subject,
'Sessions': len(subject_logs),
'Total Hours': subject_logs['study_duration'].sum() / 60,
'Avg Score': subject_logs['quiz_score'].mean()
})

subject_df = pd.DataFrame(subject_data)
st.dataframe(subject_df, use_container_width=True)
else:
st.info("►► No progress data available yet. Start logging study sessions to see analytics!")

# ========== FOOTER ==========
st.markdown("---")
st.markdown("""
<div style='text-align: center; color: #888; font-size: 0.9em;'>
<p>►► Integrated Student Analytics & Study Dashboard | Powered by Streamlit & Machine Learning</p>
<p>Monitor your performance, get personalized study plans, and achieve your academic goals!</p>
</div>
""", unsafe_allow_html=True)
```